

Lesson 3 — Drawing With Letters

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The three steps, every frame

1. **Clear** the buffer
2. **Draw** into the buffer
3. **Show** it, all at once

Nothing reaches the screen until the frame is finished. That is **double buffering**.

A grid is one long line

```
char screen[WIDTH * HEIGHT];

int index = y * WIDTH + x;    // skip y rows,
                              // then x across
int x = index % WIDTH;        // the
                              // remainder
int y = index / WIDTH;        // the whole
                              // part
```

Always bounds-check

```
void put(int x, int y, char c) {
    if (x < 0 || x >= WIDTH ||
        y < 0 || y >= HEIGHT) return;
    screen[y * WIDTH + x] = c;
}
```

Python raises IndexError. C++ does not. It writes into memory it does not own and fails somewhere else, later.

Showing it

```
std::string out = "\033[H";    // cursor
home, no clear
for (int y = 0; y < HEIGHT; y++) {
    for (int x = 0; x < WIDTH; x++)
        out += screen[y * WIDTH + x];
    out += '\n';
}
std::cout << out << std::flush;
```

`\033[2J` clears (more flicker) · `\033[H` homes (less)

Frame timing

```
using Clock = std::chrono::steady_clock;
auto start = Clock::now();
// ...do the frame...
auto elapsed = Clock::now() - start;
if (elapsed < target)
    std::this_thread::sleep_for(target -
    elapsed);
```

`steady_clock`, **not** `system_clock` — the system clock can jump.

Delta time

```
std::chrono::duration<double> d = now -
last;
double dt = d.count();
if (dt > 0.1) dt = 1.0 / 60.0;    // clamp
```

Gentle repeating motion

```
sin(time * SPEED) * SIZE
```

Input multiplier = how fast. Output multiplier = how far.

Section A — Recall

A1. What are the three steps of a frame when you use a screen buffer? What is the technique called? [3 marks]

A2. Write the formula that converts `(x, y)` into an index, and both formulas for the reverse. [3 marks]

A3. What happens in C++ if you write past the end of an array? How is that different from Python, and why is it worse? [3 marks]

A4. Why `steady_clock` rather than `system_clock`? [2 marks]

A5. In `sin(time * 3.0) * 5.0`, what does each number control? [2 marks]

Section B — Predict the output

B1. `WIDTH` is 32. What index is the square at `(5, 3)`? And which `(x, y)` is index 101? Show your working. [3 marks]

B2. `WIDTH = 32`, `HEIGHT = 7`. What does `put(4, 2, '#')` do with this formula, and what should it have done? [4 marks]

```
screen[x * WIDTH + y] = c;
```

B3. The frame's work takes 4 ms and the target is 16 ms. How long does the program sleep, and what is the resulting frame rate? Now the work takes 25 ms — what happens? [3 marks]

B4. The bat is drawn with `sin(frame * 0.9)` instead of `sin(total_time * 0.9)`. Describe what happens when `TARGET_FPS` changes from 30 to 60. What is this bug called? [4 marks]

Section C — Find and fix the bug

C1. The last column of the dungeon is never drawn. What is wrong? [3 marks]

```
for (int x = 0; x < WIDTH - 1; x++) {  
    put(x, y, DUNGEON[y][x]);  
}
```

C2. This appears to work on one student's machine, prints garbage on another's, and crashes on a third. [4 marks]
Why do all three things happen, and what should be added?

```
void put(int x, int y, char c) {  
    screen[y * WIDTH + x] = c;  
}
```

C3. The animation is a blur and the frame rate reads in the hundreds of thousands. What is missing? [3 marks]

Section D — Build it

- ☐ **Checkpoint 1.** Make a `char screen[WIDTH * HEIGHT]` buffer, a `clear()` that fills it, and a `present()` that prints it as rows. Show a screen full of dots.
- ☐ **Checkpoint 2.** Write `put(x, y, c)` **with the bounds check**. Test it by deliberately calling `put(-5, 0, 'X')` and `put(999, 0, 'X')` and checking nothing breaks.
- ☐ **Checkpoint 3.** Draw a dungeon from an array of strings. Edit the strings, recompile, and confirm the drawing code did not change.
- ☐ **Checkpoint 4.** Add the frame timer with `steady_clock` and `sleep_for`. Draw the measured frame rate on screen.
- ☐ **Checkpoint 5.** Animate something with `sin(total_time * speed)` — rippling water, a flickering torch, or a creature moving back and forth.
- ☐ **Checkpoint 6.** Change `TARGET_FPS` from 30 to 10 and back. Your animation should run at the **same real-world speed** either way. If it does not, you are driving it from the frame count.

CHECKPOINT 6 IS THE TEST THAT MATTERS

Anything driven by `frame` instead of `total_time` will speed up and slow

down with the frame rate. It is the same bug the web and Python tracks met as `<em>delta time</em>`, wearing different clothes.

Section E — Design challenge

NO RIGHT ANSWER

E1. Your buffer holds one `char` per square. What if you wanted **colour** as well? Design the data. How much memory would a 200×60 dungeon take then?

E2. You clear and redraw **every** square every frame, even though the walls never change. For 32×7 that is nothing; for 200×60 it is 12,000 characters a frame. **Design a way to redraw only what changed.** What does it cost you?

E3. You sleep for the leftover time. What happens if the frame's work takes **longer** than the budget? Does the game slow down, skip ahead, or something else? What would you *want* it to do, and is that always the same answer?

E4. You have now met double buffering three times: turtle's `tracer(0)`, the web canvas's erase-then-draw, and today building it yourself out of a plain array. **What did building it teach you that being handed it did not?**

Stretch goals

1. **Colour** with ANSI codes (`"\033[31m"` red, `"\033[0m"` reset). Careful — the codes are several characters, so they do not fit in a one-`char`-per-square buffer. How will you solve that?
2. More things to animate: a dripping ceiling, a patrolling guard, a pulsing portal.

3. **Load the dungeon from a text file** so you can edit levels without recompiling.
4. A bigger dungeon that scrolls, showing only the part near the player. That is a **camera**.